A Short Nonrelativizing Circuit Lower Bound for PP

An "Aaronsonized" proof of Vinodchandran's theorem

Abstract

For every constant c, there is a language in PP requiring Boolean circuits of size larger than n^c. The proof below isolates two ideas. First, a direct halving diagonalization, due to Aaronson, constructs a language in P^{PP} that defeats all size-n^c circuits. Second, the LFKN interactive proof for counting gives a Karp–Lipton-type collapse: if $\mathsf{P}^{\#\mathsf{P}}$ has polynomial-size circuits, then $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{MA}$. Since $\mathsf{MA} \subseteq \mathsf{PP}$ and $\mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}}$, an assumed size-$n^c$ upper bound for PP collapses the diagonal language back into PP, giving the contradiction.

The route deliberately avoids the polynomial hierarchy, Toda's theorem, and Kannan's theorem. Its only substantial imported theorem is the efficient-prover form of the LFKN protocol.

1 The theorem and the proof plan

Theorem 1.1 (Vinodchandran)**.** *For every constant $c \geq 1$,*

$$\mathsf{PP} \not\subseteq \mathsf{SIZE}(n^c).$$

Equivalently, for every fixed c, some language in PP has no family of Boolean circuits of size n^c.

The proof has three short components and one imported interactive-proof fact:

(1) *Direct diagonalization.* For every c,

$$\mathsf{P}^{\mathsf{PP}} \not\subseteq \mathsf{SIZE}(n^c).$$

The construction simply keeps the size-n^c circuits still consistent with the answers chosen so far and always answers against their majority. Each new answer kills at least half of the surviving circuits.

(2) *Counting and majority are equivalent as oracles.*

$$\mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}}.$$

(3) *Efficient-prover collapse.* The LFKN protocol implies

$$\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P/poly} \quad \Longrightarrow \quad \mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{MA}.$$

(4) Finally,

$$\mathsf{MA} \subseteq \mathsf{PP}.$$

Assuming $\mathsf{PP} \subseteq \mathsf{SIZE}(n^c)$ implies $\mathsf{P}^{\mathsf{PP}} \subseteq \mathsf{P/poly}$. By (2)–(4), this forces

$$\mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}} = \mathsf{MA} = \mathsf{PP}.$$

But (1) supplies a language in P^{PP} without size-n^c circuits. Once $\mathsf{P}^{\mathsf{PP}} = \mathsf{PP}$, this contradicts the assumption.

We now prove each step carefully.

2 Preliminaries: PP, GapP, and MA

We use the following convenient characterization of PP. A GapP function is a difference $g = f_1 - f_0$ of two #P functions. Then

$$L \in \mathsf{PP} \iff \exists g \in \mathsf{GapP} \text{ such that } x \in L \iff g(x) > 0.$$

This formulation lets us compare two exponentially large, polynomial-time definable sets directly.

We will need only one theorem about Merlin–Arthur computation.

Lemma 2.1. $\mathsf{MA} \subseteq \mathsf{PP}$.

Proof. Let $L \in \mathsf{MA}$. Suppose the verifier uses an $m(n)$-bit witness and $r(n)$ random bits. Amplify the protocol so that

$$\begin{aligned} x \in L &\implies \exists w: \quad \Pr[V(x,w;R)=1] \geq 1 - 2^{-m-2}, \\ x \notin L &\implies \forall w: \quad \Pr[V(x,w;R)=1] \leq 2^{-m-2}. \end{aligned}$$

Let $A(x,w)$ be the number of random strings on which V accepts with witness w, and set

$$S(x) = \sum_{w \in \{0,1\}^m} A(x,w).$$

This is a #P function. On a no-instance,

$$S(x) \leq 2^m \, 2^{r-m-2} = 2^{r-2},$$

whereas on a yes-instance a single good witness gives

$$S(x) \geq (1 - 2^{-m-2}) 2^r > 2^{r-1}.$$

Hence

$$g(x) := 2S(x) - 2^r$$

is positive on yes-instances and negative on no-instances. Since $g \in \mathsf{GapP}$, we have $L \in \mathsf{PP}$. □

Notice that we need no closure theorem for PP, and no facts about AM or the polynomial hierarchy.

3 Aaronson's halving diagonalization

The first ingredient is completely relativizing and elementary.

Theorem 3.1 (Aaronson)**.** *For every constant $c \geq 1$,*

$$\mathsf{P}^{\mathsf{PP}} \not\subseteq \mathsf{SIZE}(n^c).$$

Proof. Fix an input length n, and put $s = n^c$. Let $\mathcal{C}_n$ be the set of all Boolean circuits on n inputs of size at most s, using some fixed finite basis. We use fixed-length padded descriptions, so membership in $\mathcal{C}_n$, and evaluation of a described circuit, are decidable in polynomial time. Standard circuit counting gives

$$\log_2 |\mathcal{C}_n| = O(s \log s) = n^{O(1)}.$$

Set

$$N := \lceil \log_2 |\mathcal{C}_n| \rceil + 1,$$

and let $x_1, x_2, \ldots$ be the lexicographic ordering of $\{0,1\}^n$.

We define the first N values of a Boolean function L_n greedily. Let $\mathcal{C}_0 := \mathcal{C}_n$. Having defined $L_n(x_1), \ldots, L_n(x_t)$, let

$$\mathcal{C}_t := \{C \in \mathcal{C}_n : C(x_i) = L_n(x_i) \text{ for every } i \le t\}.$$

To choose $L_n(x_{t+1})$, look at the surviving circuits $\mathcal{C}_t$ and answer against their majority:

$$L_n(x_{t+1}) = \begin{cases} 0, & \#\{C \in \mathcal{C}_t : C(x_{t+1}) = 1\} \ge \#\{C \in \mathcal{C}_t : C(x_{t+1}) = 0\}, \\ 1, & \text{otherwise.} \end{cases}$$

For all $t > N$, simply set $L_n(x_t) = 0$.

By construction,

$$|\mathcal{C}_{t+1}| \le \frac{1}{2}|\mathcal{C}_t|.$$

Therefore

$$|\mathcal{C}_N| \le \frac{|\mathcal{C}_n|}{2^N} < 1,$$

so $\mathcal{C}_N = \varnothing$. No circuit of size at most n^c computes L_n.

It remains to see that the language $L = \bigcup_n L_n$ lies in P^{PP}. Given x_t, first reject immediately if $t > N$. Otherwise reconstruct $L_n(x_1), \ldots, L_n(x_t)$ sequentially. Once the first i answers are known, the difference

$$\#\{C \in \mathcal{C}_i : C(x_{i+1}) = 1\} - \#\{C \in \mathcal{C}_i : C(x_{i+1}) = 0\}$$

is a GapP function of the already known data: nondeterministically guess a circuit description, check that it is valid and agrees with the first i prescribed answers, and contribute $+1$ or -1 according to its value on x_{i+1}. A PP query therefore determines the sign of this difference, and hence the next bit of L_n.

Since $N = n^{O(1)}$, only polynomially many adaptive PP queries are needed. Thus $L \in \mathsf{P}^{\mathsf{PP}}$, while $L \notin \mathsf{SIZE}(n^c)$. □

Remark 3.2 (Why this diagonalization is unusually clean). Ordinary diagonalization against all size-n^c circuits seems to require writing down an exponentially long truth table. Here only $O(\log |\mathcal{C}_n|) = n^{O(1)}$ carefully chosen input values are needed: each one removes half of the remaining candidates. The power of PP is used only to determine which answer kills the larger half.

4 P^{PP} and $\mathsf{P}^{\#\mathsf{P}}$

Lemma 4.1.

$$\mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}}.$$

Proof. A PP query can plainly be answered with a $\#\mathsf{P}$ oracle by counting accepting paths and comparing the answer with half of all paths. Thus $\mathsf{P}^{\mathsf{PP}} \subseteq \mathsf{P}^{\#\mathsf{P}}$.

Conversely, let $f \in \#\mathsf{P}$. On inputs of length n, $f(x) \le 2^{p(n)}$ for some polynomial p. For any integer a of $p(n)+1$ bits, the threshold predicate

$$f(x) \ge a$$

lies in PP: equivalently, compare the number of accepting paths with the explicitly given threshold by adding a suitable number of dummy accepting or rejecting paths. Binary search with polynomially many such threshold queries recovers the exact value of $f(x)$. Hence $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}^{\mathsf{PP}}$. □

We henceforth write

$$\mathcal{C} := \mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}}.$$

5 Small counting circuits compress an interactive prover

The one substantial imported ingredient is the efficient-prover form of the LFKN interactive proof.

Theorem 5.1 (LFKN, efficient-prover form)**.** *Every language in $\mathsf{P}^{\#\mathsf{P}}$ has a polynomial-round interactive proof with a polynomial-time randomized verifier such that the honest prover's next-message function is computable in $\mathsf{FP}^{\#\mathsf{P}}$.*

For our application, one can obtain this directly by running the LFKN counting/permanent protocol whenever a polynomial-time machine needs a $\#\mathsf{P}$ oracle answer. The honest prover can compute the correct counting answers, and all subsequent algebraic responses, with $\#\mathsf{P}$ oracle access.

The crucial consequence is a Karp–Lipton theorem for counting computation.

Lemma 5.2 (Efficient-prover collapse)**.** *If*

$$\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}/\mathrm{poly},$$

then

$$\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{MA}.$$

Consequently,

$$\mathsf{P}^{\#\mathsf{P}} = \mathsf{MA} = \mathsf{PP}.$$

Proof. Fix $L \in \mathsf{P}^{\#\mathsf{P}}$, and take the interactive proof from Theorem 5.1. Let

$$P_{\mathrm{hon}}(x, \tau)$$

denote the honest prover's next message after transcript τ. This is a polynomial-length output computable in $\mathsf{FP}^{\#\mathsf{P}}$.

Under the hypothesis $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}/\mathrm{poly}$, every output bit of every $\mathsf{FP}^{\#\mathsf{P}}$ function has polynomial-size circuits; combining the polynomially many output-bit circuits yields a polynomial-size circuit family computing P_{hon}.

Now construct an MA protocol for L. Merlin sends Arthur a circuit D purporting to compute the prover strategy $(x, \tau) \mapsto P_{\mathrm{hon}}(x, \tau)$. Arthur then chooses all the random coins for the original interactive verifier and simulates the entire interaction, using D whenever the prover is supposed to speak.

On a yes-instance, Merlin can send the circuit for the honest prover, so completeness is inherited from the original protocol. On a no-instance, every circuit D specifies one particular prover strategy. The original interactive proof is sound against *every* prover strategy, hence in particular against the strategy represented by D. Therefore the new protocol is an MA protocol.

Thus $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{MA}$. Lemma 2.1 and the trivial containments $\mathsf{PP} \subseteq \mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}}$ give

$$\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{MA} \subseteq \mathsf{PP} \subseteq \mathsf{P}^{\#\mathsf{P}},$$

so all three classes are equal. □

Remark 5.3 (Where the proof is nonrelativizing). The halving diagonalization of Section 3 relativizes. The collapse lemma does not: it uses arithmetization in the LFKN interactive proof. This is exactly where nonrelativizing power must enter. Aaronson constructed an oracle relative to which PP has linear-size circuits, so no fully relativizing proof of Theorem 1.1 can exist.

6 Finishing the lower bound

We can now prove Theorem 1.1 in a few lines.

Proof of Theorem 1.1. Fix c, and suppose for contradiction that

$$\mathsf{PP} \subseteq \mathsf{SIZE}(n^c).$$

In particular $\mathsf{PP} \subseteq \mathsf{P/poly}$. A polynomial-time machine making polynomially many adaptive PP queries can be unrolled into a polynomial-size circuit by wiring in the appropriate polynomial-size circuits for its oracle queries. Hence

$$\mathsf{P}^{\mathsf{PP}} \subseteq \mathsf{P/poly}.$$

By Lemma 4.1, this says $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P/poly}$. The efficient-prover collapse, Lemma 5.2, therefore yields

$$\mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}} = \mathsf{MA} = \mathsf{PP}.$$

But Theorem 3.1 gives a language

$$L \in \mathsf{P}^{\mathsf{PP}} \setminus \mathsf{SIZE}(n^c).$$

Since $\mathsf{P}^{\mathsf{PP}} = \mathsf{PP}$, this language belongs to PP, contradicting our assumption that every PP language has size-n^c circuits. □

7 What is actually needed for the proof?

The argument is intentionally economical. A student needs only the following background beyond the basic circuit model.

Topic	**What is used**
PP	Majority nondeterminism or, preferably, the GapP sign characterization.
MA	Definition, amplification, and the short proof $\mathsf{MA} \subseteq \mathsf{PP}$.
$\#\mathsf{P}$	Counting accepting paths and threshold queries; enough to prove $\mathsf{P}^{\mathsf{PP}} = \mathsf{P}^{\#\mathsf{P}}$.
Interactive proofs	The LFKN counting/permanent protocol in the form "the honest prover is in $\mathsf{FP}^{\#\mathsf{P}}$."
Polynomial hierarchy	Nothing.
Toda/Kannan	Nothing.

This is one reason the proof is attractive pedagogically: after the interactive-proof machinery is in place, the circuit lower bound itself is mostly a halving argument and a one-line collapse.

8 Historical note and the promise-MA direction

Vinodchandran first proved Theorem 1.1 using the LFKN collapse together with consequences of Toda's theorem and Kannan's fixed-polynomial circuit lower bounds. Aaronson later gave the direct P^{PP} halving diagonalization used here; combining it with the classical LFKN collapse yields the streamlined proof above.

Santhanam subsequently pushed the fixed-polynomial lower bound much farther down, proving that for every c there is a promise problem in MA requiring circuits larger than n^c. That theorem needs an additional padding/self-reducibility argument and is not required here. Conceptually, however, it continues the same theme: interactive verification lets a short circuit stand in for a much more powerful prover, and circuit lower bounds emerge by forcing such a simulation to collide with diagonalization.

References

[1] S. Aaronson, *Oracles Are Subtle But Not Malicious*, Proceedings of CCC 2006; ECCC TR05-040, 2005. `https://eccc.weizmann.ac.il/report/2005/040/`

[2] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, *Algebraic Methods for Interactive Proof Systems*, Journal of the ACM 39(4):859–868, 1992.

[3] R. Santhanam, *Circuit Lower Bounds for Merlin–Arthur Classes*, SIAM Journal on Computing 39(3):1038–1061, 2009.

[4] N. V. Vinodchandran, *A Note on the Circuit Complexity of PP*, Theoretical Computer Science 347(1–2):415–418, 2005; ECCC TR04-056. `https://eccc.weizmann.ac.il/report/2004/056/`